iREX: An Interview-based Human-in-the-Loop Method for Explainable Requirements Elicitation

Anonymous author
Anonymous affiliation

Anonymous author
Anonymous affiliation

Anonymous author
Anonymous affiliation

Abstract

Early requirements documents are often underspecified, with critical decisions remaining implicit, terminology left ambiguous, and exceptional cases overlooked. To address these issues, this paper proposes iREX, a human-in-the-loop method for explainable requirements elicitation and refinement based on distinct LLM roles. The novelty of iREX lies in its controlled pipeline, which incrementally disambiguates and completes early requirements documents while ensuring that all modifications are grounded in stakeholder answers and obtained through low-effort interaction based on closed-ended clarification questions, thereby minimizing both human effort and the risk of LLM hallucinations.

iREX uses three LLM roles: *Detector*, which identifies candidate defects in the input requirements; *Question Generator* producing stakeholder-facing clarification questions, formulated as closed questions; and *Refiner* which transforms stakeholder answers into requirement revisions. Human stakeholders remain actively involved by answering these questions, with their responses recorded in a decision log. Each iteration of iREX produces (i) a clarification questionnaire, (ii) a decision log, (iii) an updated version of the requirements, and (iv) end-to-end traceability linking questions, stakeholder answers, and resulting requirement edits, supported by a Defects Register.

We report a feasibility walkthrough based on a single case study as preliminary design evidence. We also outline the empirical validation required to assess improvements in requirements quality, in terms of increased completeness and defect reduction, as well as stakeholder effort across refinements.

2012 ACM Subject Classification Software and its engineering → Requirements analysis; Computing methodologies → Artificial intelligence; Software and its engineering → Empirical software validation

Keywords and phrases Requirements refinement, ambiguity reduction, human-in-the-loop, LLM orchestration, traceability, questionnaire-based elicitation.

Digital Object Identifier 10.4230/LIPIcs.CVIT.2016.23

Acknowledgements Anonymous acknowledgements

1 Introduction

Producing high-quality requirements specifications is a challenging and delicate task in software engineering. Requirements are often incomplete, ambiguous, and evolve over time, making it difficult for analysts to capture stakeholders' intent accurately [22, 23, 30]. Consequently, early drafts of software requirements documents frequently embed implicit assumptions, omit critical details, and use vague or inconsistent terminology. Prior research in requirements engineering has extensively studied ambiguity and inconsistency detection, incompleteness identification, iterative quality refinement through inspections, structured reviews, and NLP-supported analysis [17, 5, 4, 12]. In particular, interviews are widely used in requirements elicitation to support clarification and capture stakeholder input.

LLMs offer new opportunities to support requirements engineering activities. Recent LLM-based approaches have explored requirements elicitation, question generation, interview


© Anonymous author(s);
licensed under Creative Commons License CC-BY 4.0



support, and automated refinement assistance [19, 11, 26, 25, 21, 24, 13, 7]. However, directly using LLMs for requirement generation or completion introduces significant risks, such as hallucinations, unintended scope expansion, and loss of traceability, particularly when outputs are not grounded in domain knowledge or validated by stakeholders [29, 14, 3, 16, 18]. This highlights the need for approaches that leverage LLM capabilities while preserving control, grounding, and explainability.

We focus on what we term the *last miles* of requirements engineering: the transition from a preliminary document to a final version that is complete, unambiguous, and internally consistent. This is not a simple polishing step; rather, it involves uncovering ambiguities, missing thresholds, or underspecifications, and resolving them through iterative clarification with stakeholders. In industrial practice, these refinement activities are often performed in an ad hoc and poorly documented manner, which can lead to misunderstandings, rework, limited explainability of decisions, and disputes about the rationale behind requirements changes [1, 2, 8, 9].

To address this problem, we propose **iREX**, (interactive Requirements Engineering eXplainable) an interview-based, human-in-the-loop method designed to support the last miles of requirements engineering through iterative and explainable refinement. iREX focuses on improving an initial requirements draft via structured interaction with stakeholders. The method combines Retrieval-Augmented Generation (RAG) with LLM-driven elicitation to guide the refinement process. A key goal of this work is the minimization of human involvement: stakeholders are only required to answer closed-ended clarification questions. Optionally, analysts and stakeholders may inspect, accept, correct, or reject final LLM-generated artifacts, including detected defects, requirement edits, and provenance links.

iREX takes as input an initial requirements list (e.g., a stakeholder-produced draft), together with domain-specific documentation, additional notes, and supporting web-based information. Since, in early elicitation phases, the context boundary is often unclear, RAG-derived evidence cannot be treated as fully reliable ground truth. For this reason, iREX adopts a closed-ended questionnaire strategy that keeps stakeholders actively involved in validating and constraining the refinement process. The closed-ended questionnaire strategy helps expose alternatives and exceptional cases that stakeholders may not have considered, thereby reducing the risk of overlooked requirements. By presenting informed alternatives, it also helps stakeholders overcome the "blank page" problem and make fast, low-effort decisions about aspects that would otherwise remain implicit [28, 20]. Another property of iREX is explainability: every refinement step is explicitly linked to the corresponding question and stakeholders answer, producing a transparent decision log.

This paper should be interpreted as an early-stage empirical-method contribution. The current evidence consists of a design walkthrough and preliminary feasibility observations. Its main contribution lies in defining a controlled and explainable refinement pipeline, together with measurable artifacts and evaluation dimensions for future empirical validation.

2 Proposed Approach

iREX is a human-in-the-loop requirements refinement method based on the coordinated interaction of three specialized LLM roles.

Figure 1 presents the proposed refinement loop. In each round, the current requirements list is taken as input and a revised requirements version, together with its traceability record, is produced as output. The loop is governed by a human quality gate: continuation or termination is decided by the human in the loop, when answering the generated clarification

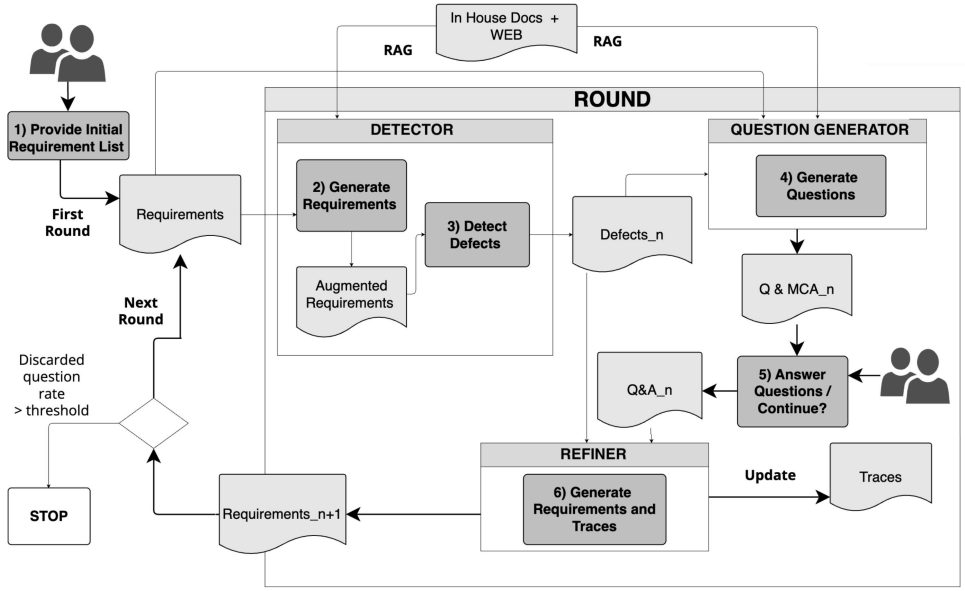


Figure 1 iREX Architecture

questions (see step 5 below).

The process instantiates agentic workflow patterns [10, 15, 27]. It follows an Orchestrator–Worker structure in which specialized components perform retrieval, defect detection, question generation, and answer-grounded revision under a fixed control flow. It uses evidence-grounded RAG to support both defect analysis and stakeholder answering.

The process is organized into the following steps:

1. The input is an initial requirements list. When needed, unstructured artifacts (e.g., draft documents) are preprocessed into an enumerated baseline list. This preprocessing is outside the scope of iREX.
2. Detector analyzes the input requirements to interpret stakeholders' needs and leverages RAG to expand them into a comprehensive requirements representation, hereafter referred to as the *augmented requirements*, which serves as a richer reference model for defect detection and coverage comparison. The augmented requirements are only a working artifact and do not constitute validated or accepted requirements.
3. Detector then analyzes the current requirements to identify candidate clarity and consistency defects and, using the *augmented requirements* as reference, potential incompleteness such as missing scenarios, edge cases, or domain-context gaps. Detected issues are recorded in a defect log and later addressed through stakeholder clarification questionnaires.
4. Question Generator generates a set of clarification questions, with multiple-choice answers and supporting evidence to help the stakeholder make an informed choice. The number of questions is dynamically bounded by the unresolved defects.
5. Stakeholders answer the questions[1] by selecting one or more options and, optionally, providing a short justification. Answers constitute the decision log and delimit requirement

[1] In the current version of iREX, stakeholders act as a single decision-making team to avoid divergent responses. Future work may distribute questions across stakeholder roles and consolidate conflicting answers.

modifications. Questions may also be discarded as irrelevant; the proportion of discarded questions is used to decide whether to continue or terminate the refinement loop.

6. Refiner translates stakeholder decisions into edits to produce the next requirements version. It also generates provenance links connecting each edit to its originating question and selected answer.

The final human acceptance of requirements is outside the scope of iREX. In practical settings, the resulting requirements would still undergo standard stakeholder and analyst validation before acceptance. In our evaluation, however, they are primarily used to assess measurable improvements in requirements quality across refinement iterations.

3 Operationalization of LLM Roles

All LLM roles use GPT-4 mini (OpenAI) with deterministic decoding (temperature = 0.0, top-p = 1.0). Some variability may still arise from retrieved external content and future model updates. Prompts are in the supplementary material: they were iteratively refined through pilot sessions to generate concise closed-ended questions and answer-grounded edits.

Detector executes two consecutive steps in each round. First, it analyzes the current Requirements$_n$ together with the retrieved context (RAG evidence) to expand the baseline into an augmented requirements view that makes implicit needs, scope items, and domain details more explicit. Second, using this augmented requirements view as a reference model, Detector (see Table 1) examines the current Requirements$_n$ to identify and record defects in the Defects register. Defects are classified using a taxonomy including ambiguity, incompleteness, and inconsistency.

Table 1 Defect Detection Prompt Excerpt

Detector Excerpt

```
You are a senior requirements engineer. Your task is to inspect the current
    requirements against the provided verified context evidence.
Do not invent requirements. Use only: (1) the current requirements, (2)
    verified evidence chunks, and (3) assumptions explicitly marked as
    assumptions.
For each candidate issue, return STRICT JSON with:
- defect_id
- target_requirement_ids
- defect_type: ambiguity | vagueness | inconsistency | incompleteness |
    missing_exception | undefined_actor | missing_parameter
- rationale
- evidence_status: verified_fact | assumption | conflicting | unsupported
- evidence_chunk_ids
- suggested_stakeholder_role
- confidence: low | medium | high
If the issue depends on an assumption, mark it as a clarification candidate,
    not as a confirmed defect.
```

To generate questions and multiple-choice answers, Question Generator (see Table 2) constructs a stakeholder questionnaire whose size is dynamically bounded by the set of unresolved defects identified in the current iteration. The Question Generator is asked to produce four domain-specific alternatives, plus *Other* (open answer), *None*, and *Useless*. This structure is designed to balance bounded decision-making with sufficient flexibility, while allowing stakeholders to reject irrelevant or overly fine-grained questions.

Refiner (Table 3) translates stakeholder answers into requirement edits. Changes are limited to what is directly implied by the selected answers, must remain within scope, avoid unnecessary design details, and preserve provenance links to the originating questions. Each edit is classified as **E** (clarification), **R** (refinement), or **N** (new within scope).

Table 2 Question Generator Prompt Excerpt

Question Generator Excerpt

```
You are an expert Requirements Engineer.

Given initial requirements, a scope coverage map, uncovered scope items -
    defects extracted from documents,  and intrinsic lexical defect candidates

Create clarification questions to remove ambiguities, clarify lexical defects,
     and uncover missing scope.

Return STRICT JSON only.
- Make questions answerable by a stakeholder.
[ ... other question formating options .. ]
```

Table 3 Refiner Answer-Grounded Revision Prompt Excerpt

Refiner Revision Prompt Excerpt

```
You are a senior requirements engineer revising an SRS.

Use only validated stakeholder decisions. Do not introduce capabilities not
    directly implied by answers. If answers conflict, create no speculative
    edit; mark the affected requirement as TBD and report the conflict.

For each proposed edit, return STRICT JSON with:
- edit_id - old_requirement_id
- new_requirement_text - edit_type: clarification | refinement | split |
    new_within_scope | TBD
- based_on_question_ids - based_on_selected_options - rationale
```

4 Envisioned Methodology: Study Design

We plan a comparative study with at least two conditions: a structured human-only requirements refinement process and a treatment condition based on the proposed iterative LLM loop. Both conditions will start from the same Context Pack and the same baseline SRS v_1, ensuring a consistent basis for comparison. The control condition will represent realistic current requirements refinement practice. Analysts will refine the requirements using their usual expertise, tools, and working habits, including any general-purpose AI tools they would normally use. They will not, however, be given access to the proposed iterative LLM loop, its prompts, templates, orchestration mechanisms, or traceability support.

Final requirements quality will be assessed independently by blinded reviewers who do not know which condition produced each artifact. The review list will include completeness, ambiguity, and consistency. A further assessor will analyse the treatment condition to assess stakeholder-intent alignment and traceability. We will report inter-rater agreement and resolve disagreements through adjudication.

4.1 Research Questions

- **RQ1 (Final requirements quality):** How does iREX affect the quality of refined requirements in terms of quality (clarity, consistency and completeness) compared with the initial list and with an expert human refinement?
- **RQ2 (Traceability and explainability):** Does end-to-end traceability from evidence and stakeholder answers to requirement edits improve perceived justification, reviewability, and stakeholder acceptance of changes?
- **RQ3 (Process feasibility and burden):** What effort and burden does iREX impose in terms of time, number of questions, discarded questions, and stakeholder workload?

4.2 Datasets and Settings

We intend to evaluate on (i) public requirements datasets–e.g. PURE [6]–for replicability, (ii) academic case studies, and (iii) industrial case studies focusing on auditability and acceptance. For public and academic case studies, stakeholder interactions will be simulated through proxy stakeholders, such as students or researchers familiar with the application domain. For industrial case studies, actual project stakeholders (e.g., analysts, domain experts, or product owners) will be involved whenever feasible to provide realistic clarification feedback and to assess the usefulness and acceptability of the generated refinements.

4.3 Measures

First, we use some simple round-level measures derived directly from the artifacts and produced at each round of the process:

- D_n: number of candidate defects ($|Defects_n|$) identified in round n.
- M_n: proportion of questions in round n that are discarded as useless by the stakeholder, i.e., $M_n = |QA_n^{discarded}|/|QA_n|$.
- R_n: total number of requirements ($|Requirements_n|$) at the end of round n.
- T_n: time spent by stakeholders

These process measures are complemented by independent human outcome measures aimed at assessing improvements in requirements quality:

- Q: improvement in requirements quality, measured as the reduction in identified quality defects (clarity, consistency, completeness) between initial and final requirement versions
- E: number of accepted/rejected edits
- Tr: traceability completeness vs number of unsupported edits (treatment condition only)

5 Proof-of-Concept Walkthrough (Illustrative)

This section illustrates feasibility of the approach, not a full empirical evaluation, by walking through some iterations with iREX. We report iteration-level measures that can be derived directly from the logs. The walkthrough is intended to illustrate how the proposed process operates in practice and to provide preliminary evidence of feasibility. It does not establish effectiveness, because it involves a single case, a single stakeholder, and no independent blinded assessment. Nevertheless, the generated questions were answered by a senior stakeholder with expertise in sailing and regatta management, serving as coordinator of a national sailing technical division. All documents are in the supplementary material.

5.1 Case-study software (Sail Cup).

The case study is a cross-platform (Android/iOS) mobile application designed to track sailing boats in real time during regattas. The baseline specification includes account registration/login, server-backed persistence of user and boat data, and periodic telemetry uploads (e.g., latitude/longitude and speed/course over ground) to a backend service. The application must also support: Wi-Fi acquisition of onboard instrument data (e.g., wind instruments, AIS), race-start assistance (countdown/flags), live race metrics, a graphical fleet view, and community-style sharing (including real-time weather sharing for safety).

5.2 Initial Requirements and In_HouseDocuments

One author, acting as the analyst and familiar with the application domain, produced the initial baseline, *Requirements_1*, from a stakeholder interview in approximately two hours. The baseline contains 30 requirements, a subset of which is shown in Table 4.

The In_House documents capture the stakeholder product vision (*DOC_A*) and the technical maritime context (*DOC_B*). *DOC_A* describes the target mobile application, including regatta scenarios, telemetry transmission, onboard Wi-Fi instrument data, and weather-sharing features. *DOC_B* provides background on the NMEA 2000 protocol, including exchanged data types and marine device communication.

Table 4 Subset of the initial requirements for the Sail Cup system

ID	Requirements
REQ-07	The system shall provide functions to create, view, and update sailors' profiles.
REQ-14	The system must assign a bow number to each boat enrolled in the regatta.
REQ-19	The mobile application should support acquiring onboard instrument data via Wi-Fi when available, including wind instruments, depth sounder data, and AIS.

5.3 Questions and Response options

Table 5 illustrates a clarification question generated for an ambiguity defect in REQ-14. The table reports the defect type, the target requirement, the generated question, the supporting evidence, and the available answer options with their rationale. For ambiguity defects, the requirement itself acts as evidence; for inconsistency defects, the evidence consists of the conflicting requirements; and for completeness defects, the evidence is derived from the retrieved RAG context.

Table 5 Q&A to resolve a lexical defect in REQ-14

Question ID: Q8 (Quantification Ambiguity) **Target:** REQ-14

REQ-14: The system must assign a bow number to each boat enrolled in the regatta.
Question: What does 'each' refer to in the context of assigning a bow number to each boat enrolled in the regatta?
Justification: The requirement REQ-14 uses the term 'each' without explicitly clarifying the intended referent, leading to lexical vagueness regarding the scope of boat identification and assignment (DEF-LEX-008).
Evidence: The system must assign a bow number to each boat enrolled in the regatta.

Option	Rationale
A) Each boat must be registered	*Registration is typically a prerequisite for participation in events like regattas.*
B) Each boat must be unique	*Unique identifiers are essential in competitive events to avoid confusion between boats.*
C) Each boat must have a profile	*Having a profile for each boat helps in managing and tracking their performance during the event.*
D) Each boat must be enrolled	*Enrollment is necessary for a boat to participate in a regatta, implying that 'each' refers to enrolled boats.*
E) Other (provide complementary details)	*Additional details could include specifications or requirements for the boats in the regatta.*
F) None	*This option suggests that 'each' does not refer to any of the previous options, which is unlikely in a structured event.*
X) Useless	

5.4 Improved Requirements

Tables 6 and 7 present answer-grounded refinements produced in the first iteration. Table 6 shows the split of REQ-5 into role-specific requirements, clarifying the responsibilities of sailors and organizers. Table 7 introduces a new requirement to address a GPS Tracking coverage gap, making explicit the system's real-time tracking, telemetry, fleet management, and GPS tracking capabilities. The tables also highlight the edit type, traceability data, and how the stakeholder decisions specify the resulting role-specific responsibilities and GPS tracking functionality. Moreover, the stakeholder-selected answer for each question is indicated with an **X** and highlighted in **bold**.

Table 6 Resolving incompleteness in REQ-5

Iteration: 1 | Edit Type: SPLIT | Question Type: Incompleteness	
Requirement	**REQ-5:** Users of the system are sailors and organizers.
Defect	**DEF-INT-LLM-009 (intrinsic):** The requirement does not specify the roles or responsibilities of sailors and organizers within the system, leaving it unclear what functionalities or interactions are expected from each user type.
Question	What roles or responsibilities do sailors and organizers have within the system?
Options	**[X] A) Sailors can manage their profiles**; **[X] B) Organizers can create and manage regattas**; **[X] C) Sailors can enroll in regattas**; D) Organizers can view telemetry data.
Edited Reqs.	**REQ-5.A:** Sailors can manage their profiles. **REQ-5.B:** Sailors can enroll in regattas. **REQ-5.C:** Organizers can create and manage regattas.

Table 7 Resolving coverage gap for GPS Tracking

Iteration: 1 | Edit Type: NEW | Question Type: Coverage	
Requirement	*No original requirement.*
Defect	**DEF-25 (coverage):** GPS Tracking is not explicitly mentioned in the requirements, although retrieved evidence indicates the relevance of real-time tracking and unauthorized use detection for the system scope.
Question	Should the requirements explicitly cover GPS Tracking functionalities?
Options	**[X] A) Yes, it should include tracking capabilities**; B) Yes, it should include alerts for unauthorized use; C) No, it is not necessary; D) No, it is covered implicitly; E) Other (provide complementary details); F) None.
Edited Reqs.	**REQ-NEW-001:** The system must provide real-time tracking and telemetry functionalities, including fleet management capabilities and GPS tracking.

5.5 Measures

The requirements refinement process for the Sail Cup system took five iterations and approximately one hour.

Table 8 D_n: Number of candidate defects distribution per iteration by type, as in the LLM log.

Iteration	**1**	**2**	**3**	**4**	**5**
Clarity defects	23	26	23	11	7
Completeness defects	36	5	3	5	3
Total candidate defects	59	31	26	16	10

In Table 8, we report the number of candidate defects identified at each iteration, distinguishing between clarity defects (ambiguity, vagueness, and inconsistency) and completeness defects. Completeness defects decrease more rapidly, especially between the first and second

iterations, because the initial RAG-supported augmentation exposes many missing scenarios and constraints early in the process. Subsequent iterations focus mainly on finer-grained clarity and consistency issues.

Table 9 reports the proportion between discarded and generated clarification questions across iterations. Generated questions match the number of detected defects, except for the first iteration, where the LLM reduced the number of generated questions from 59 candidate defects to 49 questions. Indeed, rather than enforcing a one-to-one mapping between defects and questions, Question Generator may consolidate related defects into a single question to reduce redundancy and cognitive load. Based on stakeholder feedback, most discarded questions fell into three recurring categories: (i) retrieval noise and scope creep, where RAG introduced irrelevant or out-of-scope contextual information; (ii) overly fine-grained clarification requests concerning low-impact operational details; and (iii) lexical ambiguities considered too minor to justify further refinement.

Table 9 M_n: Proportion of useless questions per iteration

Iteration	**1**	**2**	**3**	**4**	**5**
M_n	16/49	3/31	6/26	7/16	10/10

Table 10 shows the evolution of requirements across iterations: the total number grows from 30 before iteration 1 to 67 after iteration 5, mainly due to split operations, with additional contributions from a limited number of new and modified requirements in the earlier iterations.

Table 10 R_n: Requirements evolution per iteration

Iteration	**1**	**2**	**3**	**4**	**5**
$R_n \rightarrow R_{n+1}$	$30 \rightarrow 42$	$42 \rightarrow 58$	$58 \rightarrow 64$	$64 \rightarrow 67$	$67 \rightarrow 67$
$(New, Modified, Split)$	$(1, 9, 11)$	$(4, 2, 12)$	$(4, 2, 2)$	$(0, 0, 3)$	$(0, 0, 0)$

5.6 Emerging Results Summary

The walkthrough and a comparative analysis of the initial and final requirement sets suggest that iREX can support requirement refinement by preserving the core functional scope while enriching and completing functional aspects. These observations suggest promising refinement support and motivate a controlled empirical evaluation rather than constituting conclusive evidence of effectiveness. We also report the presence of two inconsistencies and a few redundancies in the final set, which suggests that additional prompt fine-tuning could further improve refinement precision and requirement consolidation.

5.7 Risks and Limitations

Question oversimplification. Some requirements decisions may not fit fixed multiple-choice answers, leading to incomplete or forced decisions. *Mitigation:* we have included `Other`, and `None` options with escalation to follow-up discussion when needed.

Question quality. Poorly formulated questions may bias stakeholder answers or propagate misunderstandings. *Mitigation:* we have included `Useless` to discard low-value questions.

Scope drift and hallucinations. RAG and LLM-generated artifacts may introduce unsupported or out-of-scope content. *Mitigation:* enforce answer-grounded edits, scope constraints, and independent human assessment of final requirements quality.

Metric validity. Process measures alone may not fully capture actual improvements in requirements quality. *Mitigation:* use complementary outcome measures and independent human assessment, as described in Section 4.3.

Evidence maturity and generalizability. The current walkthrough demonstrates feasibility rather than effectiveness, and results may vary across domains and stakeholder profiles. *Mitigation:* replicate the evaluation on public, academic, and industrial case studies.

Single-team assumption. This version assumes a single stakeholder team, which may not reflect large projects with multiple viewpoints. *Mitigation:* future work will evaluate multi-stakeholder variants with parallel answering and conflict-resolution mechanisms.

Scalability. Large requirements sets may generate too many clarification questions. *Mitigation:* prioritization, batching, question consolidation, and bounded iteration sessions.

6 Research Plan

We plan to extend the evaluation through a series of controlled experiments, each using a different baseline requirements set. In each experiment, one stakeholder team will use iREX by answering the generated clarification questions (treatment condition), while a separate team will manually refine the same baseline requirements without iREX support (control condition). Using different teams helps reduce learning and carry-over bias across conditions, while organizing experiments around independent team pairs makes recruitment more feasible. The experiments will address the research questions defined above using the measures introduced in Section 4.3, covering requirements quality, stakeholder effort, discarded questions, edits, and provenance completeness.

We will also evaluate a multi-stakeholder variant in which different stakeholders answer questions in parallel according to their expertise, with conflicts resolved through consolidation strategies and, when necessary, additional clarification questions.

We will replicate the study on public, academic, and industrial case studies to assess robustness across domains, baseline quality levels, and project settings, including scalability with respect to requirements size, generated questions, and iteration effort.

7 Conclusions

In this paper, we presented iREX, a human-in-the-loop method for explainable requirements elicitation and refinement, targeting the *last miles* of requirements engineering. iREX supports the transition from an initial requirements draft to a more explicit and traceable version through an iterative, questionnaire-driven workflow. The method combines LLM-based candidate defect detection, RAG-supported contextualization, and answer-grounded requirement updates. The novelty of iREX lies in operationalizing requirements refinement as a controlled loop in which heterogeneous project artifacts, including emails, notes, and technical documentation, are used to inform clarification questions, while requirement edits remain constrained by explicit stakeholder answers. In this way, RAG is not used as ground truth, but to support the identification of alternatives that are then validated by humans.

At this stage, iREX should be considered an emerging-results contribution rather than a validated solution. The walkthrough provides preliminary feasibility evidence. Future work will focus on controlled empirical evaluation, including independent assessment of final requirements quality, comparison with human refinement baselines, multiple case studies and stakeholders. Overall, iREX provides an initial operational framework for studying structured, stakeholder-grounded, LLM-supported requirements refinement.

Data Availability Statement

The dataset supporting this study, including the mining scripts, raw repository data, and analysis notebooks, is openly available on Zenodo at https://doi.org/10.5281/zenodo.20412689 under a CC-BY 4.0 license. The replication package contains all materials necessary to reproduce our findings.

References

1 Marcello M. Bersani, Matteo Camilli, Livia Lestingi, Raffaela Mirandola, Matteo Rossi, and Patrizia Scandurra. A conceptual framework for explainability requirements in software-intensive systems. In *2023 IEEE 31st International Requirements Engineering Conference Workshops (REW)*, pages 309–315, 2023.

2 Fabiano Dalpiaz, Ivor Van Der Schalk, and Garm Lucassen. Pinpointing ambiguity and incompleteness in requirements engineering via information visualization and NLP. In *Requirements Engineering: Foundation for Software Quality - 24th International Working Conference, REFSQ 2018*, volume 10753 of *LNCS*, pages 119–135. Springer, 2018. doi:10.1007/978-3-319-77243-1_8.

3 Moemen Ebrahim, Shawkat Guirguis, and Christine Basta. Enhancing software requirements engineering with language models and prompting techniques: Insights from the current research and future directions. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 4: Student Research Workshop)*, pages 486–496, 2025.

4 Saad Ezzini, Sallam Abualhaija, Chetan Arora, and Mehrdad Sabetzadeh. Automated handling of anaphoric ambiguity in requirements: A multi-solution study. In *Proceedings of the 44th International Conference on Software Engineering (ICSE)*, pages 187–199, 2022.

5 Henning Femmer, Daniel Méndez Fernández, Stefan Wagner, and Sebastian Eder. Rapid quality assurance with requirements smells. *Journal of Systems and Software*, 123:190–213, 2017.

6 Alessio Ferrari, Giorgio Oronzo Spagnolo, and Stefania Gnesi. Pure: A dataset of public requirements documents. In *2017 IEEE 25th International Requirements Engineering Conference (RE)*, pages 502–505. IEEE, 2017.

7 Xavier Franch, Stefania Gnesi, Federico Paccosi, Carme Quer, and Laura Semini. Leveraging requirements elicitation through software requirement patterns and llms. In *Requirements Engineering: Foundation for Software Quality - 31st International Working Conference, REFSQ 2025, Barcelona, Spain, April 7-10, 2025, Proceedings*, volume 15588 of *Lecture Notes in Computer Science*, pages 261–276. Springer, 2025.

8 Emanuele Gentili and Davide Falessi. Characterizing requirements smells. In *International Conference on Product-Focused Software Process Improvement*, pages 387–398. Springer, 2023.

9 Vincenzo Gervasi and Didar Zowghi. Reasoning about inconsistencies in natural language requirements. *ACM Transactions on Software Engineering and Methodology (TOSEM)*, 14(3):277–330, 2005.

10 Antonio Gullì. *Agentic Design Patterns: A Hands-On Guide to Building Intelligent Systems*. Springer, 2025. doi:10.1007/978-3-032-01402-3.

11 Binnur Görer and Fatma Başak Aydemir. GPT-Powered Elicitation Interview Script Generator for Requirements Engineering Training. In *32nd IEEE International Requirements Engineering Conference (RE)*, pages 372–379, 2024. doi:10.1109/RE59067.2024.00044.

12 Irit Hadar, Anna Zamansky, and Daniel M. Berry. The inconsistency between theory and practice in managing inconsistency in requirements engineering. *Empir. Softw. Eng.*, 24(6):3972–4005, 2019.

13 Junda He, Christoph Treude, and David Lo. LLM-based multi-agent systems for software engineering: Literature review, vision, and the road ahead. *ACM Transactions on Software Engineering and Methodology*, 34(5):1–30, 2025.

14 Arshia Hemmat, Mohammadreza Sharbaf, Shekoufeh Kolahdouz-Rahimi, Kevin Lano, and Sobhan Y Tehrani. Research directions for using LLM in software requirement engineering: A systematic review. *Frontiers in Computer Science*, 7:1519437, 2025.

15 Soodeh Hosseini and Hossein Seilani. The role of agentic ai in shaping a smart future: A systematic review. *Array*, 26:100399, 2025. doi:10.1016/j.array.2025.100399.

16 Lei Huang, Weijiang Yu, Weitao Ma, Weihong Zhong, Zhangyin Feng, Haotian Wang, Qianglong Chen, Weihua Peng, Xiaocheng Feng, Bing Qin, et al. A survey on hallucination in large language models: Principles, taxonomy, challenges, and open questions. *ACM Transactions on Information Systems*, 43(2):1–55, 2025.

17 Erik Kamsties, Daniel M Berry, Barbara Paech, E Kamsties, DM Berry, and B Paech. Detecting ambiguities in requirements documents using inspections. In *Proceedings of the first workshop on inspection in software engineering (WISE'01)*, volume 13, 2001.

18 Robin Korfmann, Patrick Beyersdorffer, Rainer Gerlich, Jürgen Münch, and Marco Kuhrmann. Overcoming data shortage in critical domains with data augmentation for natural language software requirements. *Journal of Software: Evolution and Process*, 37(5):e70027, 2025.

19 Alexander Korn, Samuel Gorsch, and Andreas Vogelsang. LLMREI: Automating Requirements Elicitation Interviews with LLMs. In *33rd IEEE International Requirements Engineering Conference (RE)*, pages 19–30, 2025. doi:10.1109/RE63999.2025.00013.

20 Jon A. Krosnick. Survey research. *Annual Review of Psychology*, 50:537–567, 1999.

21 Dipeeka Luitel, Shabnam Hassani, and Mehrdad Sabetzadeh. Improving requirements completeness: Automated assistance through large language models. *Requirements Engineering*, 29(1):73–95, 2024.

22 Hendrik Meth, Manuel Brhel, and Alexander Maedche. The state of the art in automated requirements elicitation. *Information and Software Technology*, 55(10):1695–1709, 2013.

23 Bashar Nuseibeh and Steve M. Easterbrook. Requirements engineering: a roadmap. In Anthony Finkelstein, editor, *22nd International Conference on Software Engineering (ICSE), Future of Software Engineering Track*, pages 35–46. ACM, 2000. doi:10.1145/336512.336523.

24 Marc Oriol, Quim Motger, Jordi Marco, and Xavier Franch. Multi-agent debate strategies to enhance requirements engineering with large language models. In *2025 IEEE 33rd International Requirements Engineering Conference (RE)*, pages 527–534. IEEE, 2025.

25 Gokul Rejithkumar, Preethu Rose Anish, Jyoti Shukla, and Smita Ghaisas. Probing with precision: Probing question generation for architectural information elicitation. In *Proceedings of the 1st IEEE/ACM Workshop on Multi-disciplinary, Open, and RElevant Requirements Engineering*, pages 8–14, 2024.

26 Krishna Ronanki, Christian Berger, and Jennifer Horkoff. Investigating ChatGPT's potential to assist in requirements elicitation processes. In *2023 49th Euromicro conference on software engineering and advanced applications (SEAA)*, pages 354–361. IEEE, 2023.

27 Aditi Singh, Abul Ehtesham, Saket Kumar, and Tala Talaei Khoei. Enhancing AI Systems with Agentic Workflows Patterns in Large Language Model. In *2024 IEEE World AI IoT Congress (AIIoT)*, pages 527–532, 2024. doi:10.1109/AIIoT61789.2024.10578990.

28 Roger Tourangeau, Lance J. Rips, and Kenneth Rasinski. *The Psychology of Survey Response*. Cambridge University Press, 2000.

29 A. Vogelsang and J. Fischbach. Using Large Language Models for Natural Language Processing Tasks in Requirements Engineering: A Systematic Guideline. In A. Ferrari and G. Deshpande, editors, *Handbook of Natural Language Processing for Requirements Engineering*. Springer International Publishing, Cham, Switzerland, 2025.

30 Didar Zowghi and Chad Coulin. Requirements elicitation: A survey of techniques, approaches, and tools. In Aybüke Aurum and Claes Wohlin, editors, *Engineering and Managing Software Requirements*. Springer, 2005. doi:10.1007/3-540-28244-0_2.